

CÁU TRÚC DỮ LIỆU CÂY (TREE)

ptbao@sgu.edu.vn

■ Giới thiệu CTDL cây:

Cây là một CTDL biểu diễn các mô hình phân cấp như :

- Sơ đồ tổ chức công ty
- Mục lục của sách
- Cấu trúc thư mục trong DOS/WIN/LINUX ...
- ...

Ban giám đốc

Tài vụ

Nhân sự

Kinh doanh

Thu

Chi

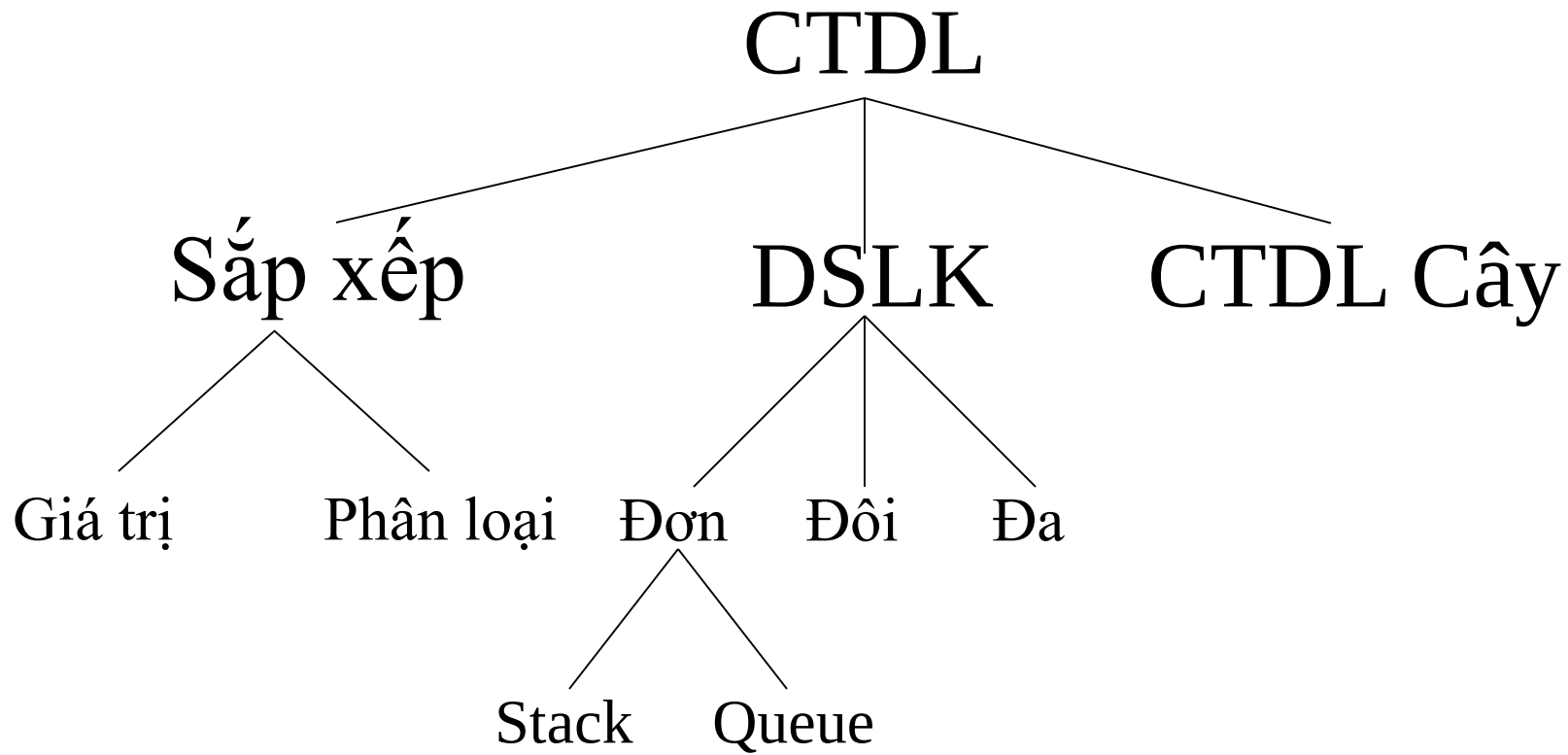
Mua

Bán

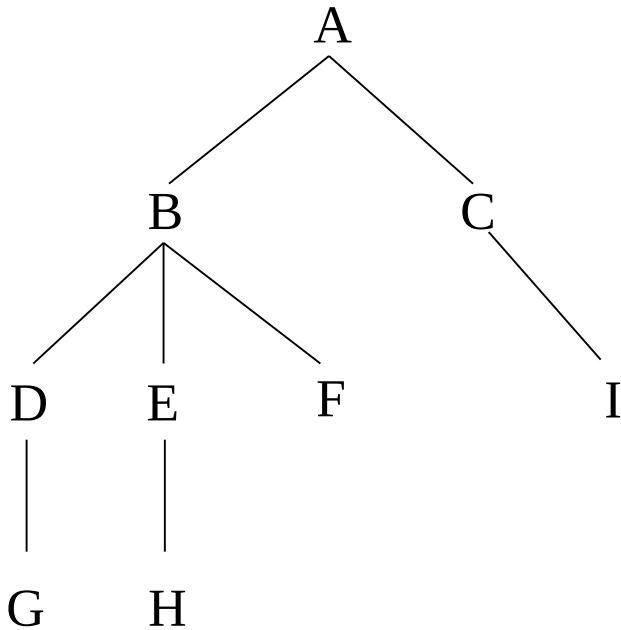
Đất

Nhà

Nhà



Một số thuật ngữ



- ✓ Cây gồm tập các node. Hai node nối với nhau bởi 01 cạnh.
- ✓ Node A gọi là node gốc.
- ✓ B là cha của D, E, và F
- ✓ C là anh em của B
- ✓ G, H, F, và I là node ngoài, hay node lá.
- ✓ A, B, C, D, và E là node trong.
- ✓ Cây có chiều cao $h = 4$
- ✓ Bậc của node B là $d(B) = 3$
- ✓ Node E nằm ở độ sâu 2 hay có mức là 2.
- ✓ Cây có N node thì sẽ có N-1 cạnh

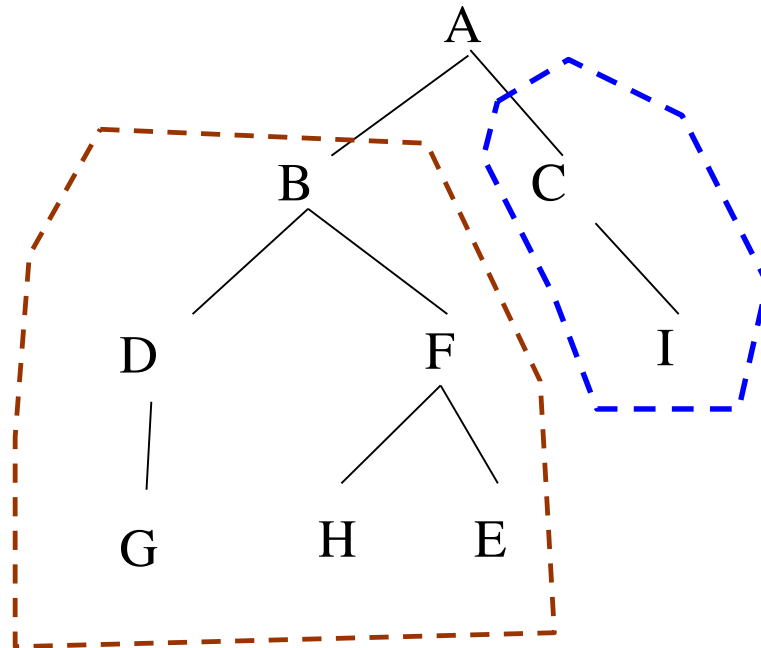
- **Nhược điểm:**

- Khi thêm phần tử, ta sẽ thêm bất kỳ, nên khi quản lý sẽ khó khăn.
- Bậc các node trên cây giao động trong biên độ lớn nên rất khó khăn việc quản lý.
- Việc xoá sẽ mất nhiều thời gian tìm kiếm và khó khăn tổ chức lại khi xoá node trong.

CÂY NHỊ PHÂN (BINARY TREE)

■ Định nghĩa:

1. Không đệ quy: Cây nhị phân là cây mà mỗi node có bậc tối đa là 2.
2. Theo đệ quy:
 - Hoặc là một node lá,
 - Hoặc cây gồm 01 node gốc và 02 cây nhị phân con của nó (cây con trái và cây con phải).



■ Tính chất:

1. Số node nằm ở mức $i \leq 2^i$
2. Số node lá $\leq 2^{h-1}$ với h là chiều cao của cây.
3. Chiều cao của cây $h \geq \log_2(N)$ với N là số node trong cây.
4. Số node trong cây $\leq 2^h - 1$

- Cài đặt bằng cấu trúc trong C:

```
typedef struct tagNode{  
    DataType          Data;  
    tagNode           *Left;  
    tagNode           *Right;  
};
```

```
typedef tagNode *BinaryTree;
```

- Cài đặt bằng đối tượng trong C:

```
class tagNode{  
    friend class BinaryTree;  
    private:  
        DataType    Data;  
        tagNode      *Left,*Right;  
public:  
    tagNode(Data d=giá trị mặc định)  
        {Left=Right=NULL;Data = d}  
    ~tagNode()  
        {if(Left)delete Left;if(Right)delete Right;}  
};  
typedef tagNode *Node;
```

```
class BinaryTree{  
    private:  
        Node Root;  
    public:  
        BinaryTree(){Root=NULL;}  
        ~BinaryTree(){if(Root)delete Root;}  
        // Các phương thức thêm, xoá, tìm kiếm,  
        // đếm, ...  
};
```

- Các thao tác thông thường:
 1. Kiểm tra cây rỗng
 2. Kiểm tra 01 node có phải là node lá không ?
 3. Duyệt cây
 4. Tìm kiếm 01 phần tử

■ Duyệt cây

1. Duyệt theo thứ tự trước (NLR)

Thuật toán NLR(T)

Input: Cây nhị phân T

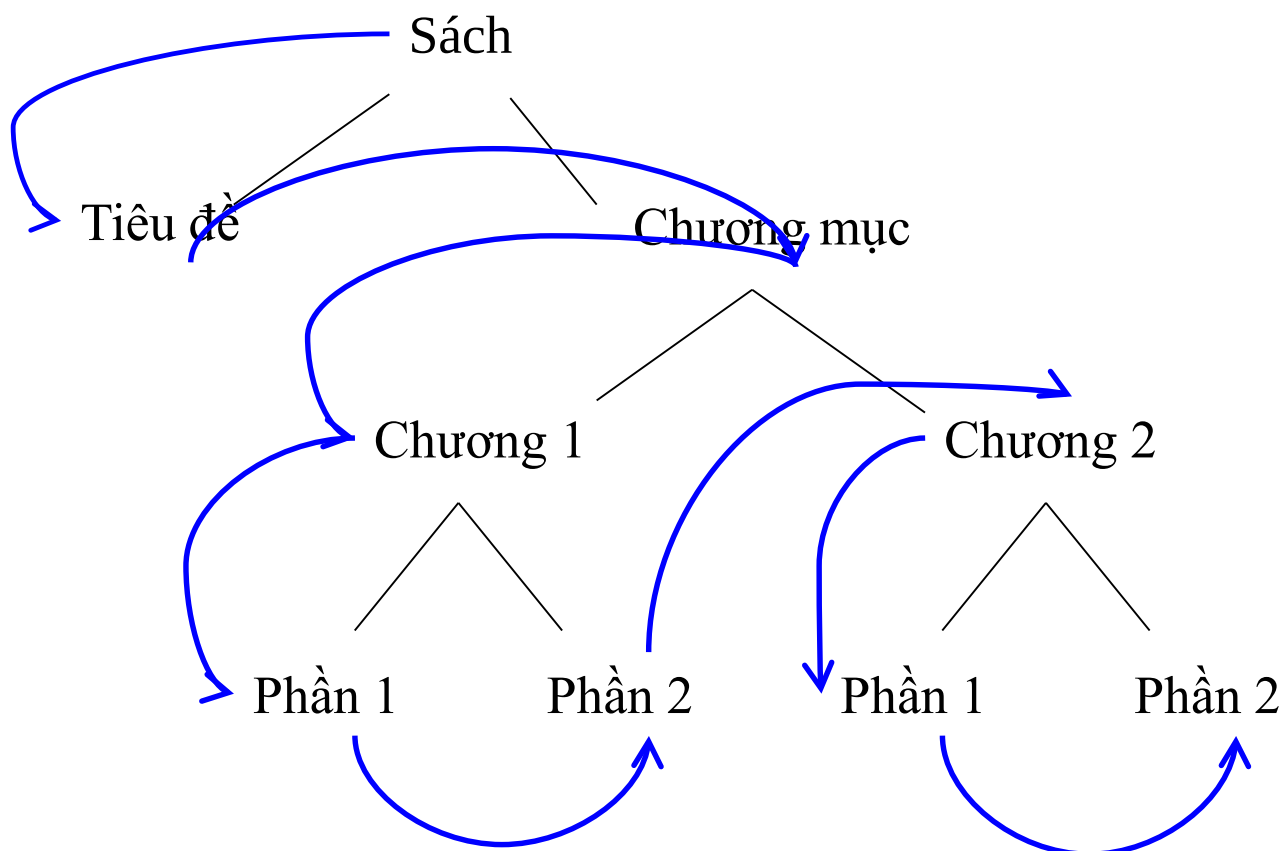
Output: tùy nhu cầu

“Thăm” node gốc T.

NLR(Con trái của T);

NLR(Con phải của T);

End.



2. Duyệt theo thứ tự sau (LRN)

Thuật toán LRN(T)

Input: Cây nhị phân T

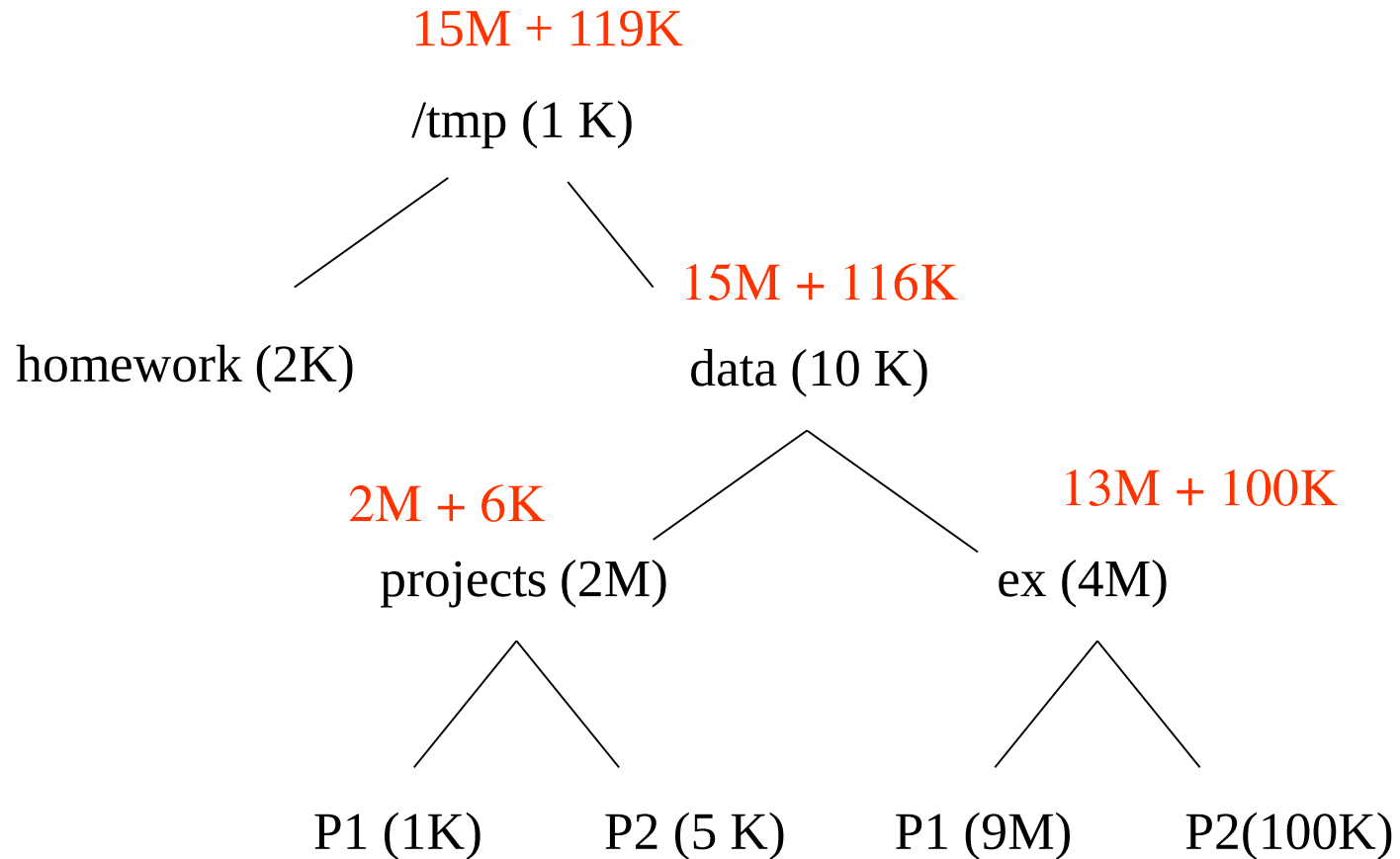
Output: tùy nhu cầu

LRN(Con trái của T);

LRN(Con phải của T);

“Thăm” node gốc T.

End.



3. Duyệt theo thứ tự giữa (LNR)

Thuật toán LNR(T)

Input: Cây nhị phân T

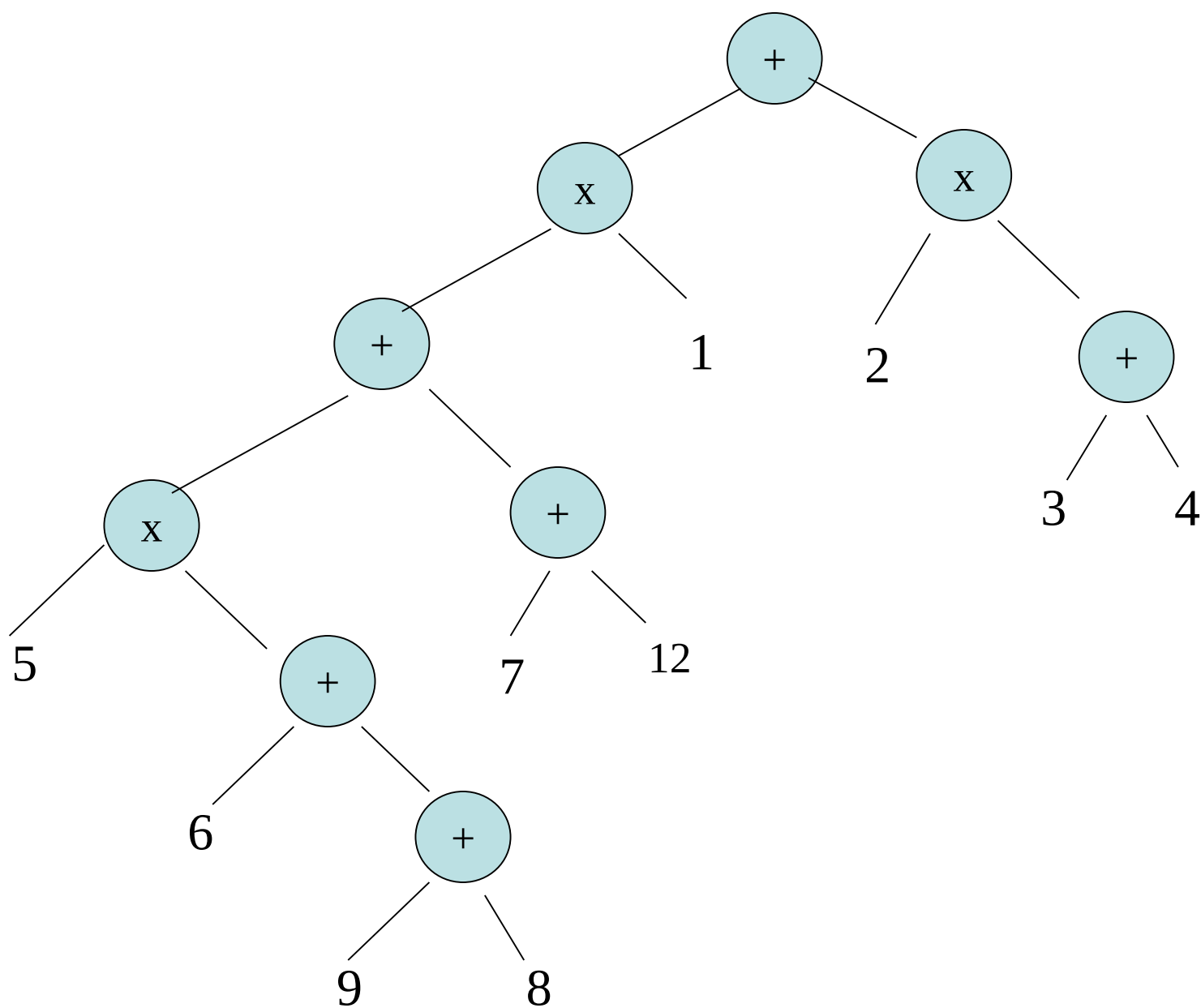
Output: tùy nhu cầu

LNR(Con trái của T);

“Thăm” node gốc T.

LNR(Con phải của T);

End.



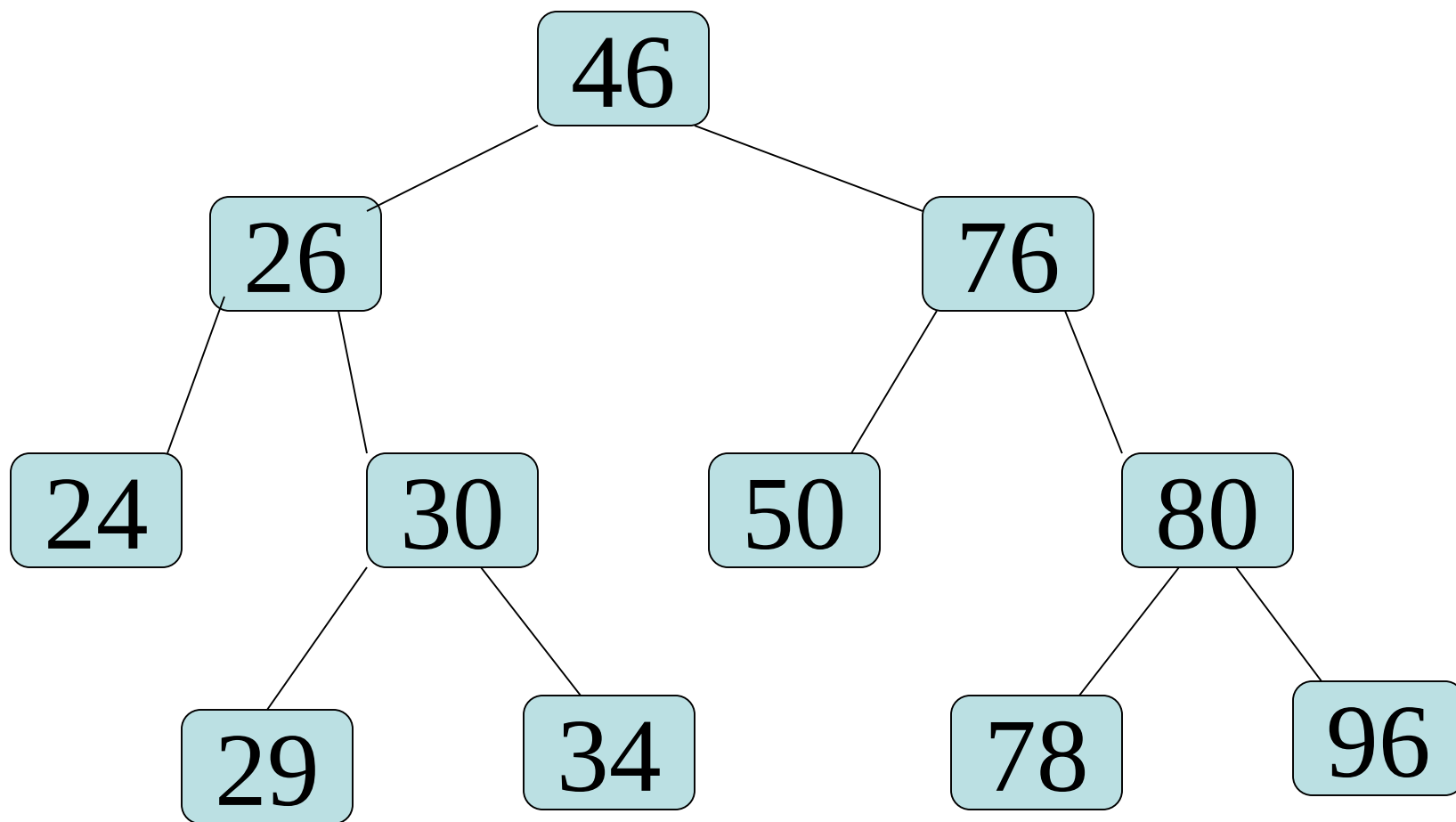
$(((((5 \times (6 + (9 + 8)))) + (7 + 12)) \times 1) + (2 \times (3 + 4)))$

- Đôi khi chúng ta quan tâm đến cả quan hệ giữa phần tử con và phần tử cha phần tử, nên chúng ta sẽ thêm 01 con trỏ Parent để biết quan hệ con cha.
- Còn khó khăn khi quản lý cây nhị phân.

CÂY NHỊ PHÂN
TÌM KIẾM
(BINARY SEARCH
TREE)

- Định nghĩa:

Là cây nhị phân mà *khoá của node bất kỳ trên cây sẽ lớn hơn khoá các node trên cây con trái và nhỏ hơn khoá các node trên cây con phải của nó.*



- Các thao tác:

1. Duyệt cây
2. Tìm kiếm 01 phần tử
3. Thêm 01 phần tử
4. Huỷ 01 phần tử
5. Huỷ cây

- Duyệt thứ tự trước:

```
void NLR(BSTree T){  
    if(T){  
        “Thăm gốc T”; // printf(“%d”,T-  
>Data);  
        NLR(T->Left);  
        NLR(T->Right);  
    }  
}
```

46, 26, 24, 30, 29, 34, 76, 50, 80, 78, 96

- Duyệt thứ tự sau:

```
void LRN(BSTree T){  
    if(T){  
        LRN (T->Left);  
        LRN (T->Right);  
        “Thăm gốc T”; // printf(“%d”,T-  
>Data);  
    }  
}
```

24, 29, 34, 30, 26, 50, 78, 96, 80, 76, 46

- Duyệt thứ tự giữa:

```
void LNR (BSTree T){  
    if(T){  
        LNR(T->Left);  
        “Thăm gốc T”; // printf(“%d”,T-  
>Data);  
        LNR(T->Right);  
    }  
}
```

24, 26, 29, 30, 34, 46, 50, 76, 78, 80, 96

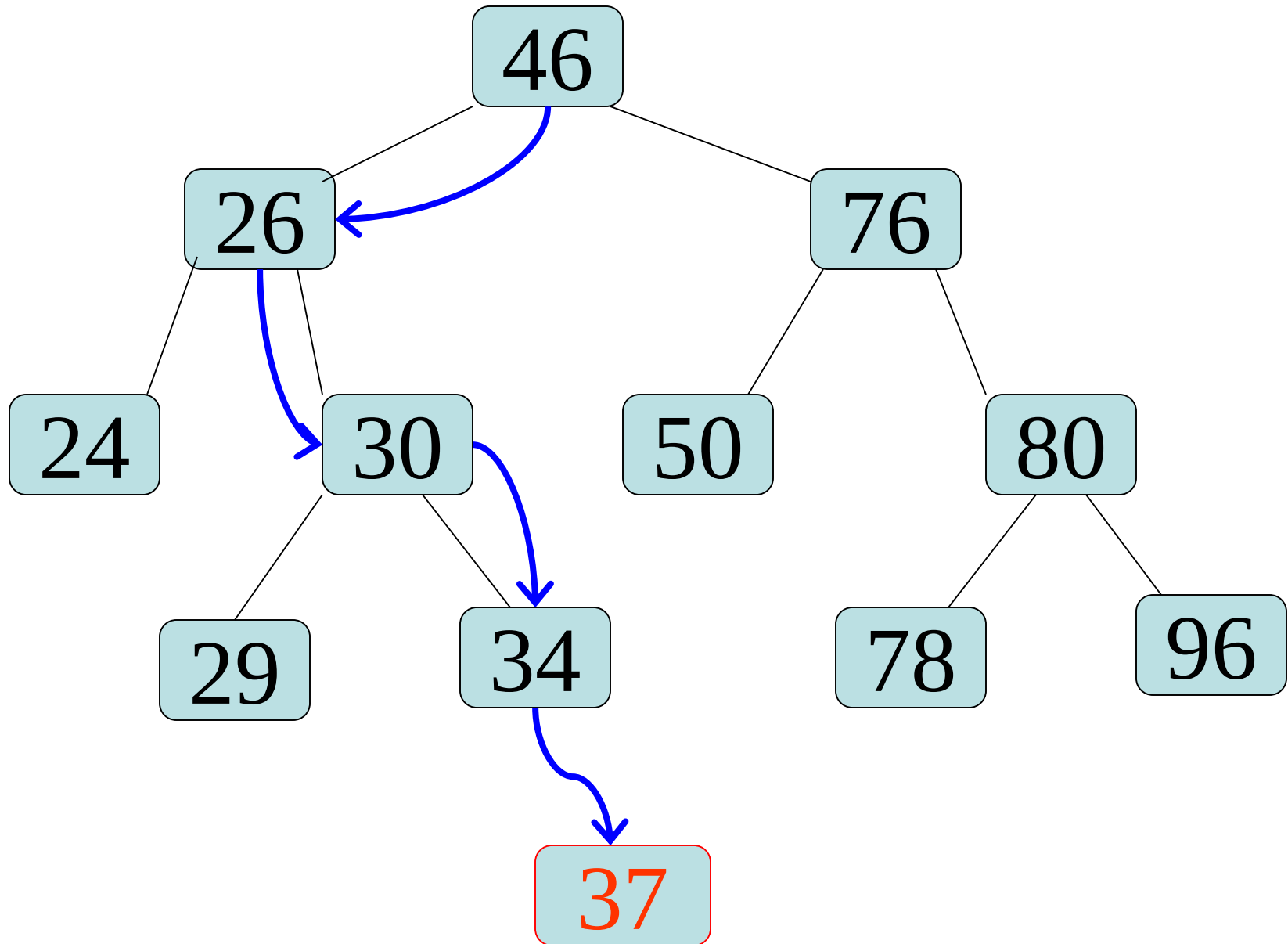
- Tìm kiếm 01 phần tử:

```
tagNode * searchNode(BSTree T, DataType x){  
    if(T){  
        if(T->Data bằng x) return T;  
        if(T->Data lớn hơn x)  
            return searchNode(T->Left,x);  
        else  
            return searchNode(T->Right,x)  
    }  
    return NULL;  
}
```

- Thêm 01 phần tử vào CNPTK:
 - ✓ Đảm bảo điều kiện ràng buộc của CNPTK.
 - ✓ Có thể thêm vào nhiều chỗ khác nhau trên cây, nhưng nếu thêm ở node lá thì sẽ dễ nhất; vì quá trình sẽ tương tự tìm kiếm.
 - ✓ Hàm insertNode sẽ trả ra giá trị: -1, 0 , 1 thể hiện không đủ bộ nhớ, giá trị thêm đã có trong CNPTK rồi, và thêm thành công.

```
int insertNode(BSTree &T, DataType x){  
    if(T){  
        if(T->Data bằng x)return 0;  
        if(T->Data lớn hơn x)  
            return insertNode(T->Left,x);  
        else  
            return insertNode(T->Right,x);  
    }  
    T = new tagNode;  
    if(!T) return -1;  
    T->Data = x;  
    T->Left = T->Right = NULL;  
    return 1;  
}
```

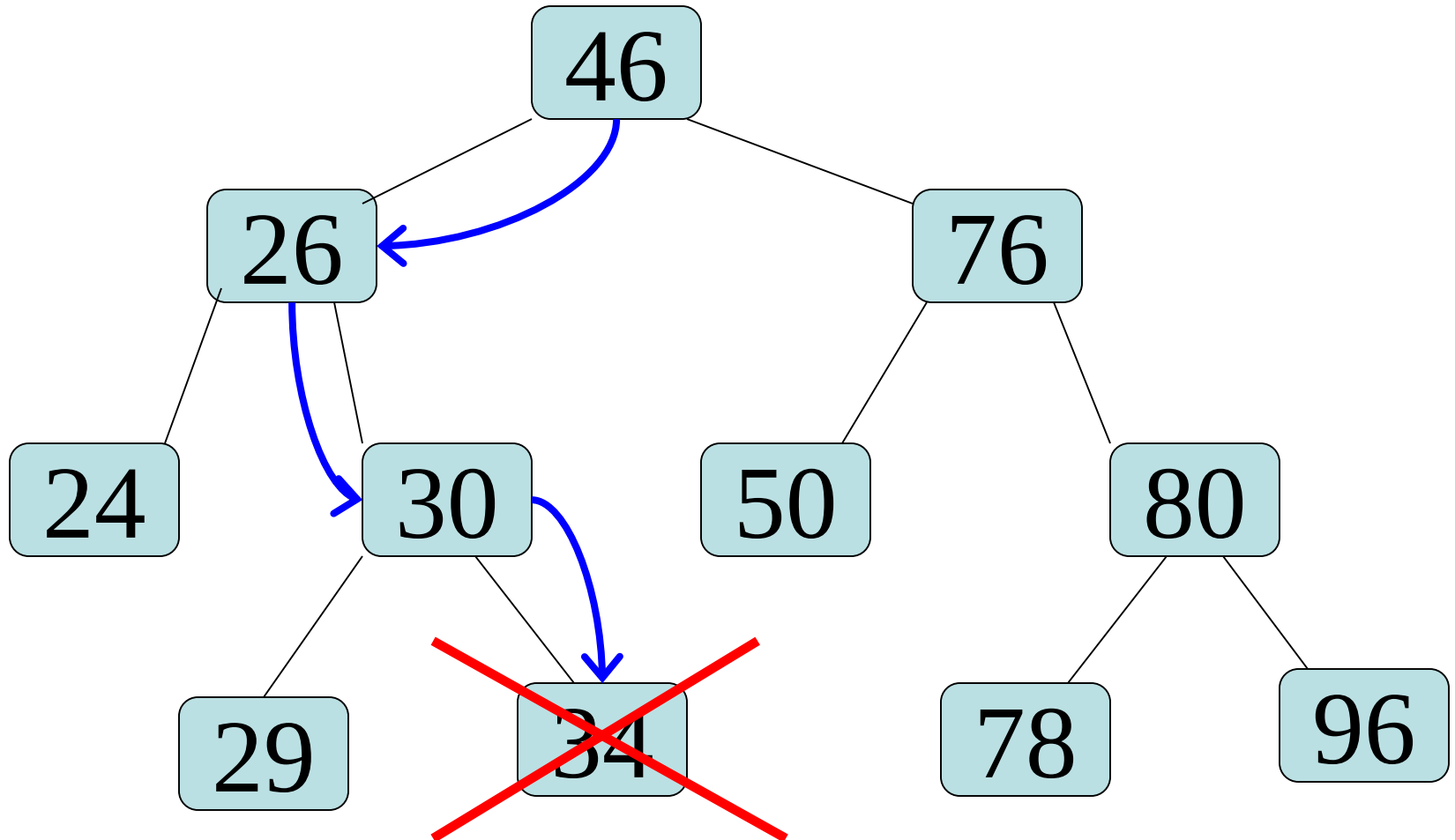
Thêm $x = 37$



- Hủy 01 phần tử trên CNPTK:
 - Đảm bảo điều kiện ràng buộc của CNPTK.
 - Có 03 trường hợp xảy ra:
 - ✓ x là node lá
 - ✓ x chỉ có 01 con (trái hay phải)
 - ✓ x có đủ 02 con

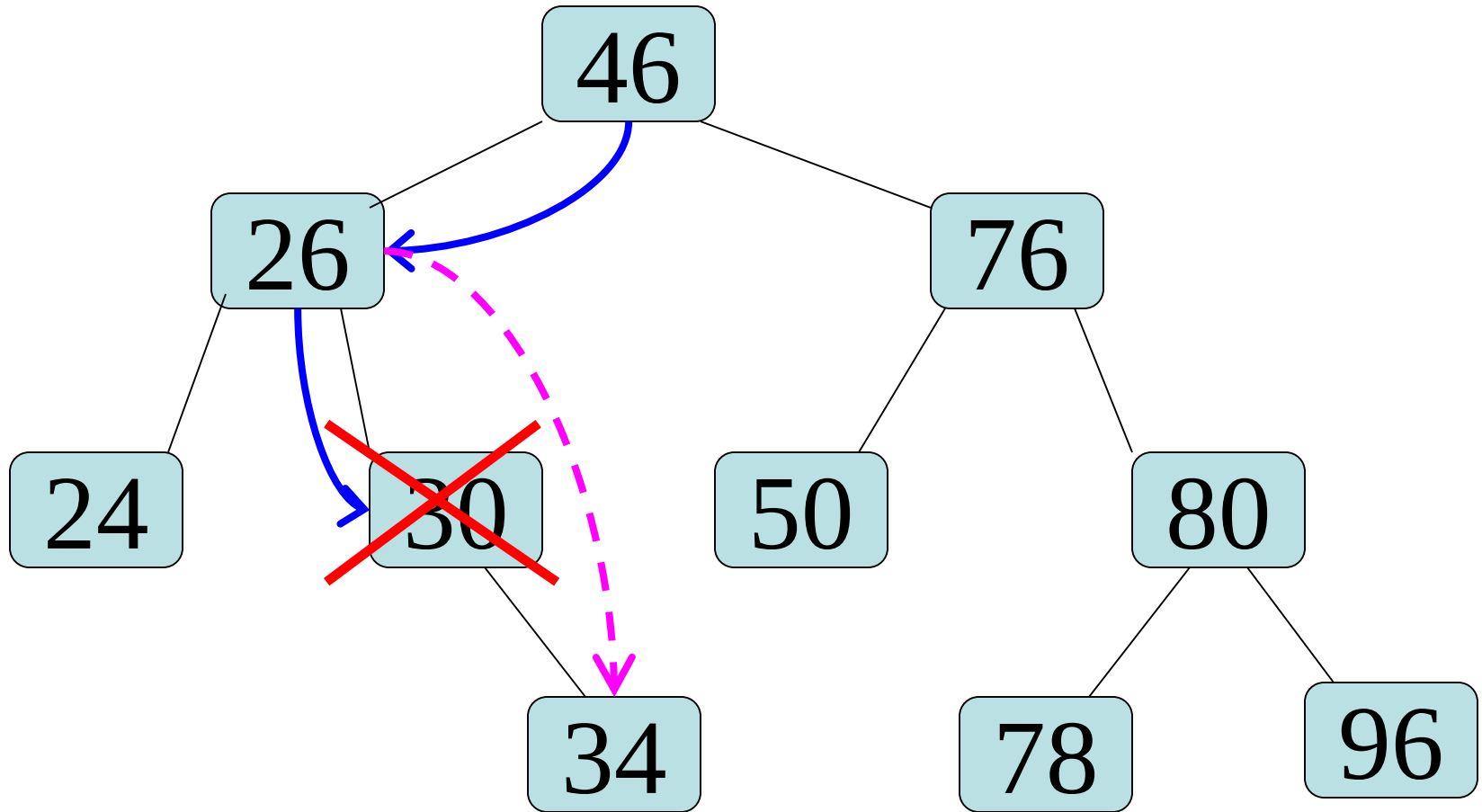
Trường hợp thứ nhất: đơn giản vì không có móc nối đến phần tử nào khác

Hủy $x = 34$



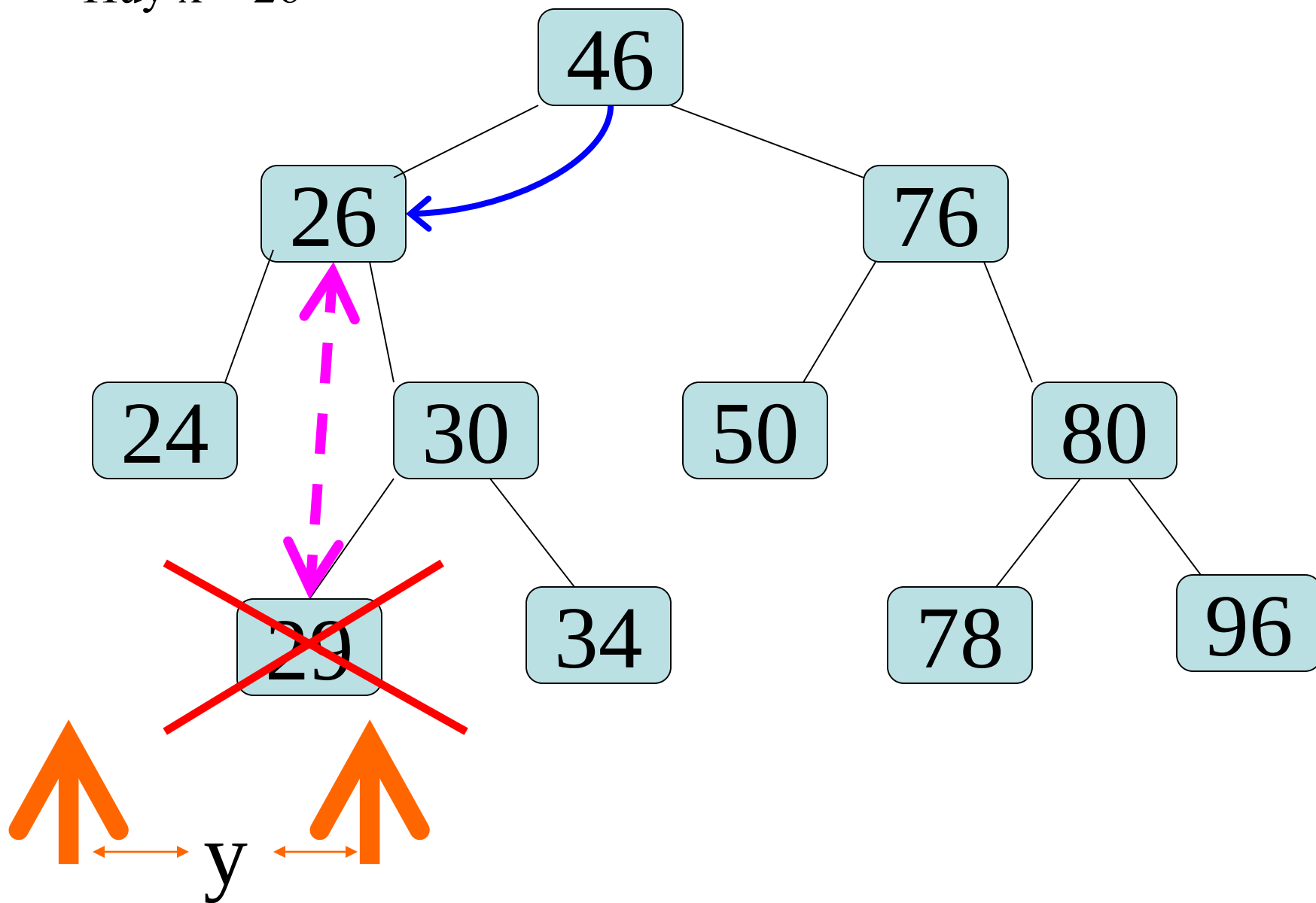
Trường hợp thứ hai: trước khi huỷ x ta móc nối cha của x và con duy nhất của nó.

Huỷ $x = 30$



- Trường hợp cuối cùng: ta không thể huỷ trực tiếp x do x có đủ 02 con (vì tái tổ chức các mối nối sẽ phức tạp), nên ta sẽ huỷ gián tiếp. Thay vì huỷ x , ta sẽ tìm 01 phần tử thế mạng y , mà phần tử này có tối đa 01 con. Thông tin lưu tại y sẽ được chuyển lưu tại x . Sau đó sẽ huỷ thật sự y đi, như 02 trường hợp đầu.
- Vấn đề đặt ra bây giờ là làm sao chọn y mà khi chuyển dữ liệu từ y lên x vẫn đảm bảo tính chất của CNPTK.
- Vậy có 02 phần tử sẽ thoả mãn đó là:
 - Phần tử nhỏ nhất trên cây con phải.
 - Hay phần tử lớn nhất trên cây con trái.

Hủy $x = 26$



```

int deleteNode(BSTree &T, DataType x){
    if(!T)return 0;
    if(T->Data lớn hơn x)
        return deleteNode(T->Left,x);
    if(T->Data nhỏ hơn x)
        return deleteNode(T->Right,x);
    else{
        tagNode *p = T;
        if(!T->Left) T = T->Right;
        else
            if(!T->Right)T = T->Left;
            else searchStandFor(p,T->Right);
        delete p;
        return 1;
    }
}

```

// phần tử nhỏ nhất trên cây con phải

```
void searchStandFor(BSTree &p, BSTree &q){  
    if(q->Left)searchStandFor(p,q->Left);  
    else{  
        p->Data = q->Data;  
        p = q;  
        q = q->Right;  
    }  
}
```

- Tạo CNPTK:
 - ✓ Lặp lại quá trình thêm 01 phần tử vào CNPTK.
- Huỷ toàn bộ cây NPTK
 - ✓ Thông qua thao tác duyệt theo thứ tự sau.
 - ✓ Vậy có thể huỷ cây theo thứ tự giữa hoặc trước được không ?

```
void removeTree(BSTree &T){  
    if(T){  
        removeTree(T->Left);  
        removeTree(T->Right);  
        delete T;  
    }  
}
```

■ Tính chất:

1. Tất cả thao tác tìm kiếm, thêm, huỷ trên CNPTK đều có độ phức tạp $O(h)$.
2. Trong trường hợp tốt nhất, CNPTK có N node sẽ có độ cao $h = \log_2(N)$, chi phí tìm kiếm khi đó sẽ tương đương tìm kiếm nhị phân trên mảng có thứ tự.
3. Tuy nhiên trong trường hợp xấu nhất, cây có thể suy biến thành 01 DSLK, khi đó các thao tác trên sẽ có độ phức tạp $O(N)$.

Bài tập

1. Đếm cây NPTK có bao nhiêu phần tử ?
2. Đếm cây NPTK có bao nhiêu node lá ?
3. Tính chiều cao cây NPTK.
4. Đếm có bao nhiêu node có 02 con
5. Đếm có bao nhiêu node có 01 con bên trái
6. Tại mức k có bao nhiêu phần tử
7. Có bao nhiêu node cha của các node lá

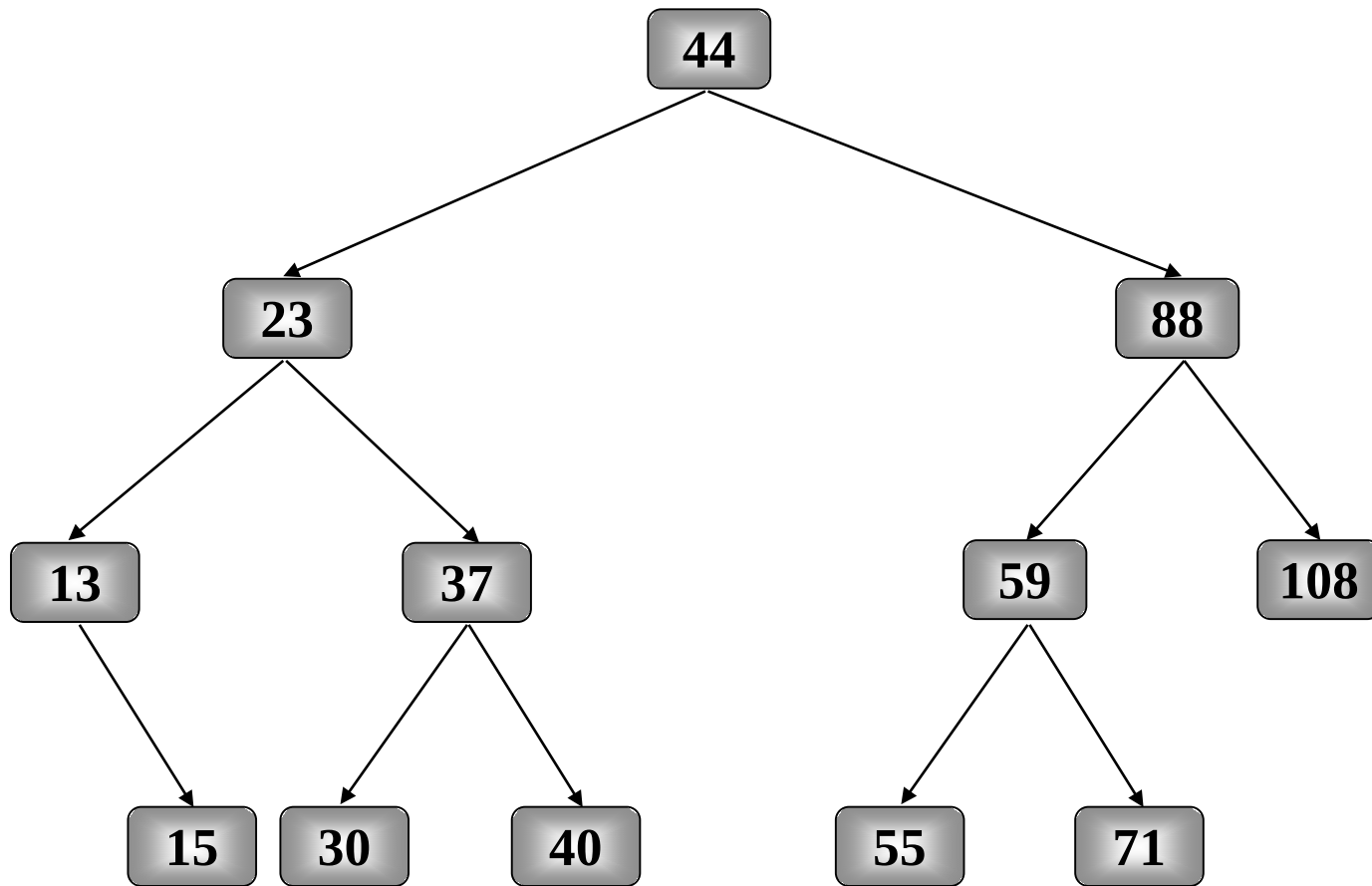
```
int heightOfTree(Node n)
{
    if(n) {
        int l = heightOfTree(n->Left);
        int r = heightOfTree(n->Right);
        return ((l>r)?l:r)+1;
    }
    return 0;
}
```

CÂY NHỊ PHÂN CÂN BẰNG (AVL TREE)

Định nghĩa

- Cây nhị phân tìm kiếm cân bằng là cây mà tại mỗi nút của nó độ cao của cây con trái và của cây con phải chênh lệch không quá một.

Cây AVL



Cây AVL

- Lịch sử cây cân bằng (AVL Tree):
 - AVL là tên viết tắt của các tác giả người Nga đã đưa ra định nghĩa của cây cân bằng Adelson-Velskii và Landis (1962).
 - Từ cây AVL, người ta đã phát triển thêm nhiều loại CTDL hữu dụng khác như cây đỏ-đen (Red-Black Tree), B-Tree, ...
- Cây AVL có chiều cao $O(\log_2(n))$.

Cây AVL

- Chỉ số cân bằng của một nút:
- Định nghĩa: Chỉ số cân bằng của một nút là hiệu của chiều cao cây con phải và cây con trái của nó.
- Đối với một cây cân bằng, chỉ số cân bằng (CSCB) của mỗi nút chỉ có thể mang một trong ba giá trị sau đây:
 - $CSCB(p) = 0 \Leftrightarrow \text{Độ cao cây trái } (p) = \text{Độ cao cây phải } (p)$
 - $CSCB(p) = 1 \Leftrightarrow \text{Độ cao cây trái } (p) < \text{Độ cao cây phải } (p)$
 - $CSCB(p) = -1 \Leftrightarrow \text{Độ cao cây trái } (p) > \text{Độ cao cây phải } (p)$

- Để tiện trong trình bày, chúng ta sẽ ký hiệu như sau: $p \rightarrow \text{balFactor} = \text{CSCB}(p)$;
- Độ cao cây trái (p) ký hiệu là h_L
- Độ cao cây phải(p) ký hiệu là h_R

Định nghĩa

```
#define LH    -1    /* Cây con trái cao hơn */
#define EH    0     /* Hai cây con bằng nhau */
#define RH    1     /* Cây con phải cao hơn */

typedef struct tagAVLNode {
    char balFactor; // Chỉ số cân bằng
    Data          key;
    struct tagAVLNode* pLeft;
    struct tagAVLNode* pRight;
}AVLNode;

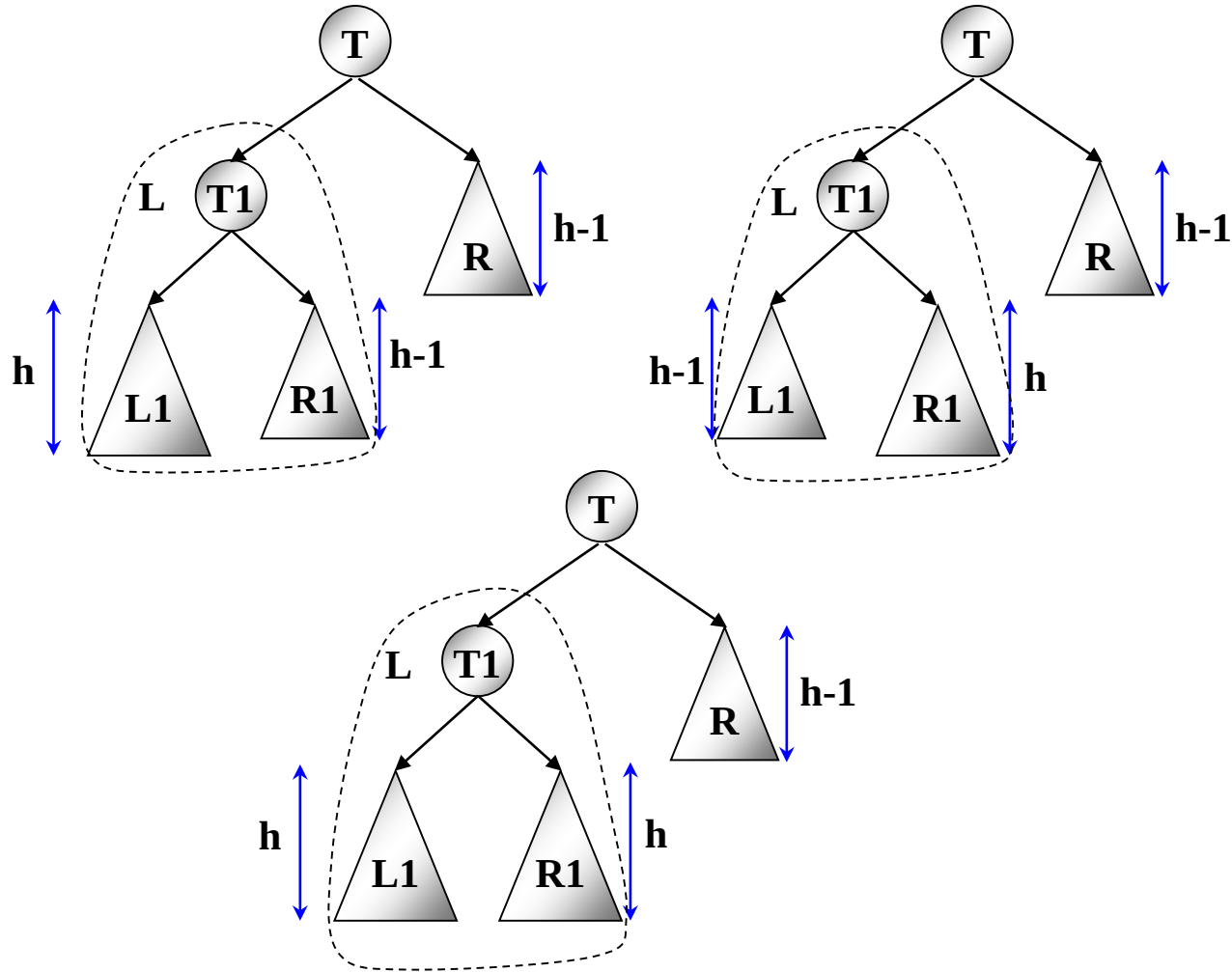
typedef AVLNode    *AVLTree;
```

- Trường hợp thêm hay hủy một phần tử trên cây có thể làm cây tăng hay giảm chiều cao, khi đó phải cân bằng lại cây.
- Việc cân bằng lại một cây sẽ phải thực hiện sao cho chỉ ảnh hưởng tối thiểu đến cây nhằm giảm thiểu chi phí cân bằng.
- Các thao tác đặc trưng của cây AVL:
 - Thêm một phần tử vào cây AVL.
 - Hủy một phần tử trên cây AVL.
 - Cân bằng lại một cây vừa bị mất cân bằng.

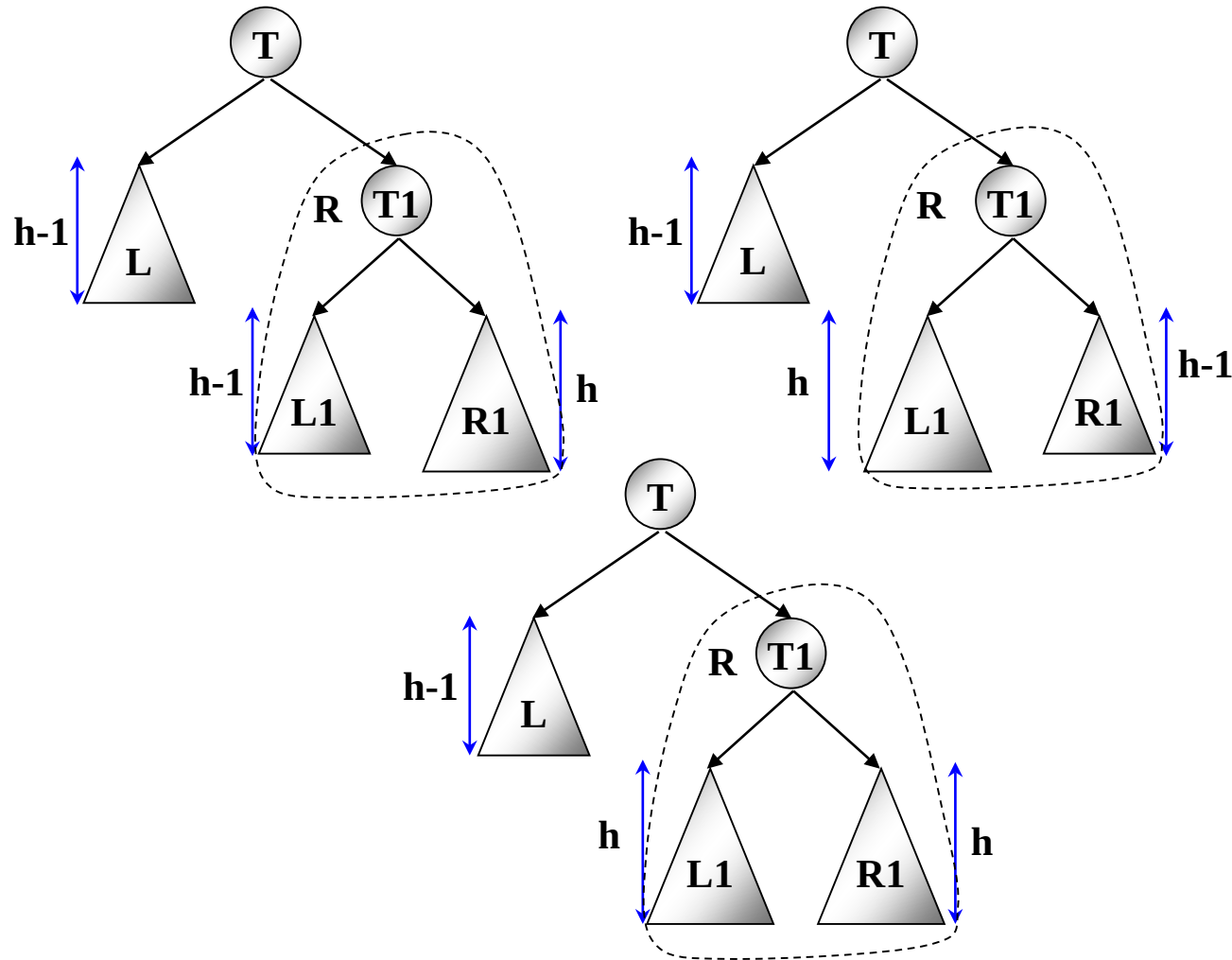
Các trường hợp mất cân bằng

- Ta sẽ không khảo sát tính cân bằng của 1 cây nhị phân bất kỳ mà chỉ quan tâm đến các khả năng mất cân bằng xảy ra khi thêm hoặc hủy một nút trên cây AVL
- Như vậy, khi mất cân bằng, độ lệch chiều cao giữa 2 cây con sẽ là 2
- Có 6 khả năng sau:
 - Trường hợp 1 - Cây T lệch về bên trái : 3 khả năng
 - Trường hợp 2 - Cây T lệch về bên phải: 3 khả năng

Trường hợp 1: cây T lệch về bên trái



Trường hợp 2: cây T lệch về bên phải

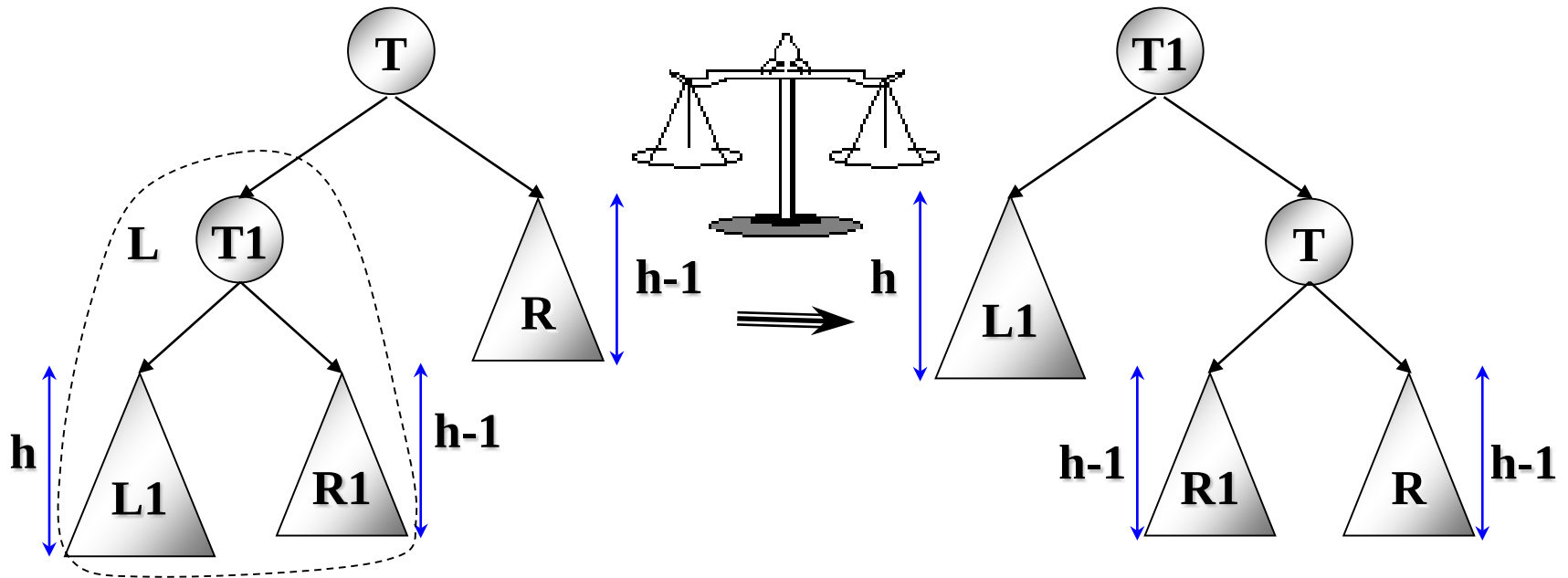


Các trường hợp mất cân bằng

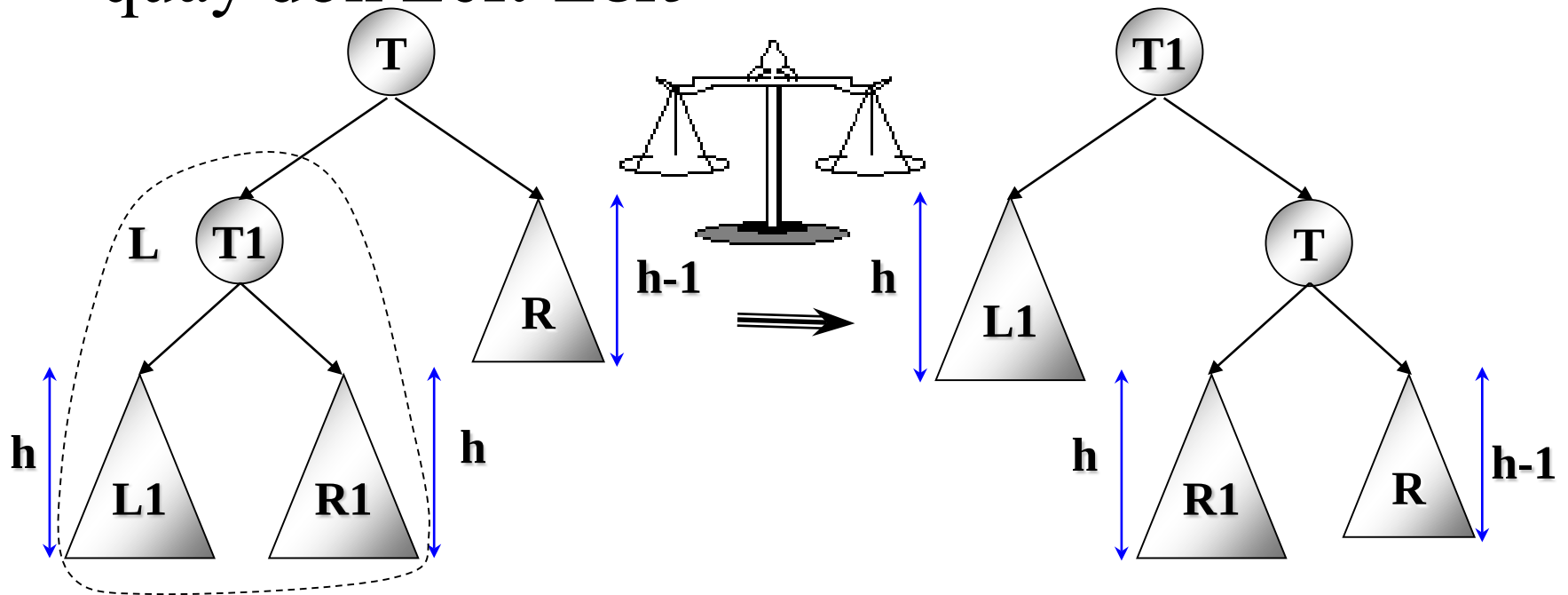
- Các trường hợp lệch về bên phải hoàn toàn đối xứng với các trường hợp lệch về bên trái.
- Vì vậy, chỉ cần khảo sát trường hợp lệch về bên trái.
- Trong 3 trường hợp lệch về bên trái, trường hợp T1 lệch phải là phức tạp nhất. Các trường hợp còn lại giải quyết rất đơn giản.

Cân bằng lại cây AVL

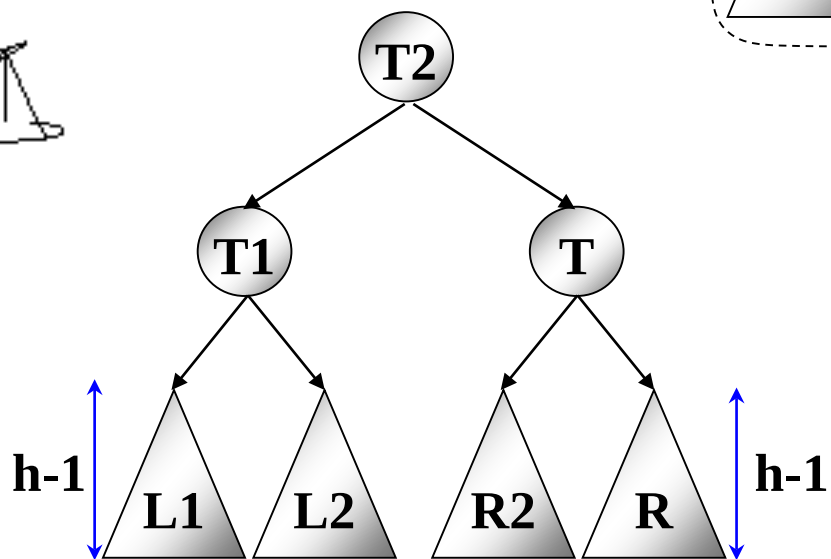
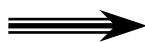
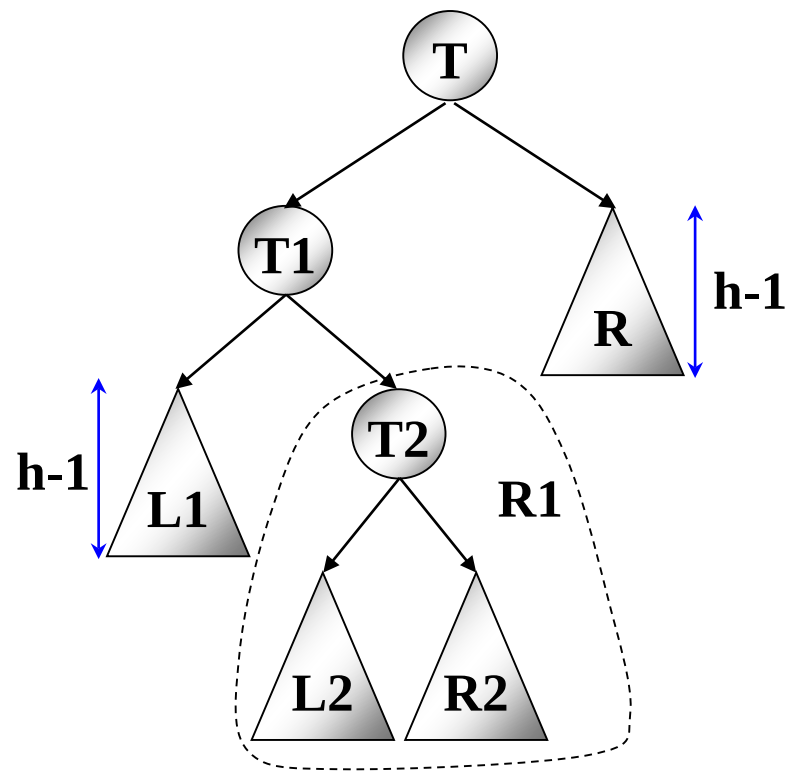
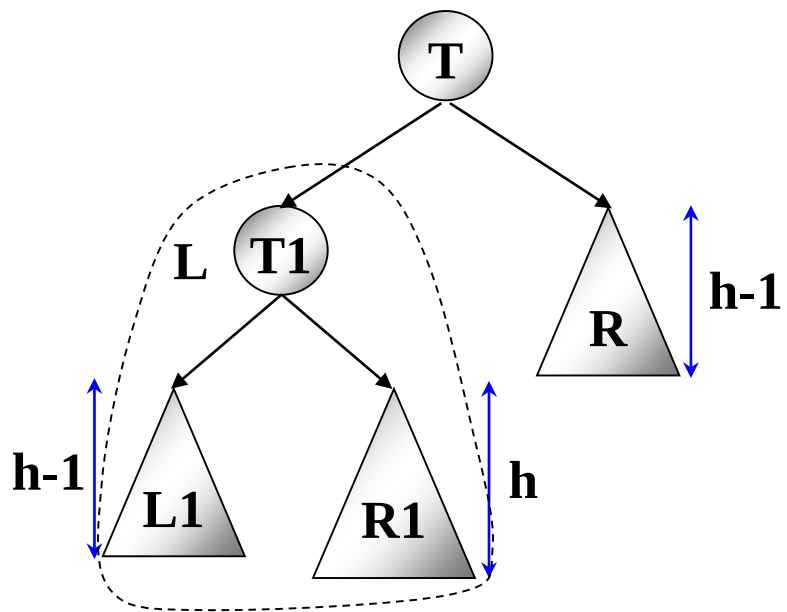
- T/h 1.1: cây T1 lệch về bên trái. Ta thực hiện phép quay đơn Left-Left



- T/h 1.2: cây T1 không lệch. Ta thực hiện phép quay đơn Left-Left



- T/h 1.3: cây T1 lệch về bên phải. Ta thực hiện phép quay kép Left-Right
- Do T1 lệch về bên phải ta không thể áp dụng phép quay đơn đã áp dụng trong 2 trường hợp trên vì khi đó cây T sẽ chuyển từ trạng thái mất cân bằng do lệch trái thành mất cân bằng do lệch phải ? cần áp dụng cách khác.

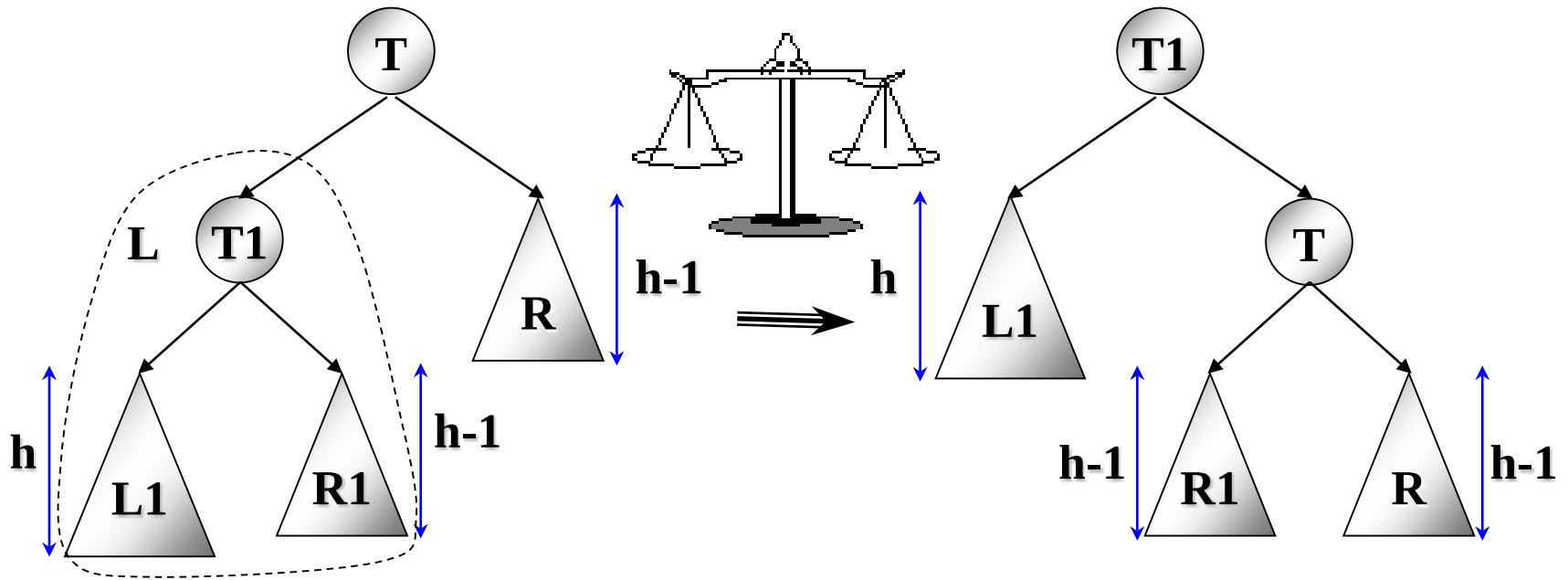


■ Lưu ý:

- Trước khi cân bằng cây T có chiều cao $h+2$ trong cả 3 trường hợp 1.1, 1.2 và 1.3.
- Sau khi cân bằng,
 - ✓ Trường hợp 1.1 và 1.3 cây có chiều cao $h+1$
 - ✓ Trường hợp 1.2 cây vẫn có chiều cao $h+2$. Đây là trường hợp duy nhất sau khi cân bằng nút T cũ có chỉ số cân bằng $\neq 0$.
 - ✓ Thao tác cân bằng lại trong tất cả các trường hợp đều có độ phức tạp $O(1)$.

Cân bằng lại cây AVL

- T/h 1.1: cây T1 lệch về bên trái. Ta thực hiện phép quay đơn Left-Left



Quay đon Left-Left

```
void rotateLL(AVLTree &T) //quay đon Left-Left
```

```
{
```

```
    AVLNode* T1 = T->pLeft;
```

```
    T->pLeft      = T1->pRight;
```

```
    T1->pRight    = T;
```

```
    switch(T1->balFactor) {
```

```
    case LH:      T->balFactor = EH;
```

```
                  T1->balFactor = EH;
```

```
                  break;
```

```
    case EH:      T->balFactor = LH;
```

```
                  T1->balFactor = RH;
```

```
                  break;
```

```
    }
```

```
    T = T1;
```

```
}
```

Quay đơn Right-Right

```
void rotateRR(AVLTree &T) //quay đơn Right-Right
{
    AVLNode* T1 = T->pRight;
    T->pRight    = T1->pLeft;
    T1->pLeft    = T;
    switch(T1->balFactor) {
        case RH:      T->balFactor = EH;
                      T1->balFactor= EH;
                      break;
        case EH:      T->balFactor = RH;
                      T1->balFactor= LH;
                      break;
    }
    T = T1;
}
```

Quay kép Left-Right

```
void rotateLR(AVLTree &T) //quay kép Left-Right
{
    AVLNode*    T1 = T->pLeft;
    AVLNode*    T2 = T1->pRight;
    T->pLeft     = T2->pRight;
    T2->pRight   = T;
    T1->pRight   = T2->pLeft;
    T2->pLeft    = T1;
    switch(T2->balFactor)    {
        case LH: T->balFactor = RH; T1->balFactor = EH; break;
        case EH: T->balFactor = EH; T1->balFactor = EH; break;
        case RH: T->balFactor = EH; T1->balFactor = LH; break;
    }
    T2->balFactor = EH;
    T = T2;
}
```

Quay kép Right-Left

```
void rotateRL(AVLTree &T) //quay kép Right-Left
{
    AVLNode*    T1 = T->pRight;
    AVLNode*    T2 = T1->pLeft;
    T->pRight    = T2->pLeft;
    T2->pLeft    = T;
    T1->pLeft    = T2->pRight;
    T2->pRight   = T1;
    switch(T2->balFactor)    {
        case RH: T->balFactor = LH; T1->balFactor = EH; break;
        case EH: T->balFactor = EH; T1->balFactor = EH; break;
        case LH: T->balFactor = EH; T1->balFactor = RH; break;
    }
    T2->balFactor = EH;
    T    = T2;
}
```


Cân bằng khi cây bị lệch về bên trái

```
int balanceLeft(AVLTree &T)
```

```
//Cân bằng khi cây bị lệch về bên trái
```

```
{
```

```
    AVLNode*    T1 = T->pLeft;
```

```
    switch(T1->balFactor)        {
```

```
    case LH:        rotateLL(T); return 2;
```

```
    case EH:        rotateLL(T); return 1;
```

```
    case RH:        rotateLR(T); return 2;
```

```
    }
```

```
    return 0;
```

```
}
```

Cân bằng khi cây bị lệch về bên phải

```
int balanceRight(AVLTree &T )  
//Cân bằng khi cây bị lệch về bên phải  
{  
    AVLNode*   T1 = T->pRight;  
  
    switch(T1->balFactor)        {  
    case LH:      rotateRL(T); return 2;  
    case EH:      rotateRR(T); return 1;  
    case RH:      rotateRR(T); return 2;  
    }  
    return 0;  
}
```

Thêm một phần tử trên cây AVL

- Việc thêm một phần tử vào cây AVL diễn ra tương tự như trên CNPTK.
- Sau khi thêm xong, nếu chiều cao của cây thay đổi, từ vị trí thêm vào, ta phải lần ngược lên gốc để kiểm tra xem có nút nào bị mất cân bằng không. Nếu có, ta phải cân bằng lại ở nút này.
- Việc cân bằng lại chỉ cần thực hiện 1 lần tại nơi mất cân bằng.
- Hàm `insertNode` trả về giá trị `-1`, `0`, `1` khi không đủ bộ nhớ, gặp nút cũ hay thành công. Nếu sau khi thêm, chiều cao cây bị tăng, giá trị `2` sẽ được trả về
- `int insertNode(AVLTree &T, DataType X)`

Thêm một phần tử trên cây AVL

```
int insertNode(AVLTree &T, DataType X)
```

```
{  int   res;
```

```
  if (T)
```

```
  {    if (T->key == X) return 0; //đã có
```

```
    if (T->key > X)
```

```
    {   res   = insertNode(T->pLeft, X);
```

```
      if(res < 2) return res;
```

```
      switch(T->balFactor)
```

```
      {    case RH: T->balFactor = EH; return 1;
```

```
        case EH: T->balFactor = LH; return 2;
```

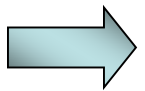
```
        case LH: balanceLeft(T);   return 1;
```

```
      }
```

```
    }
```

```
.....
```

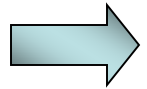
```
}
```



insertNode2

Thêm một phần tử trên cây AVL

```
int insertNode(AVLTree &T, DataType X)
{
.....
    else //  $T \rightarrow key < X$ 
    {
        res = insertNode(T-> pRight, X);
        if(res < 2) return res;
        switch(T->balFactor)
        {
            case LH: T->balFactor = EH; return 1;
            case EH: T->balFactor = RH; return 2;
            case RH: balanceRight(T); return 1;
        }
    }
.....
}
```



insertNode3

Thêm một phần tử trên cây AVL

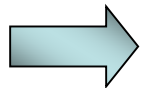
```
int insertNode(AVLTree &T, DataType X)
{
.....
    T    = new TNode;
    if(T == NULL) return -1; //thiếu bộ nhớ
    T->key = X;
    T->balFactor    = EH;
    T->pLeft = T->pRight = NULL;
    return 2; // thành công, chiều cao tăng
}
```

Hủy một phần tử trên cây AVL

- Cũng giống như thao tác thêm một nút, việc hủy một phần tử X ra khỏi cây AVL thực hiện giống như trên CNPTK.
- Sau khi hủy, nếu tính cân bằng của cây bị vi phạm ta sẽ thực hiện việc cân bằng lại.
- Tuy nhiên việc cân bằng lại trong thao tác hủy sẽ phức tạp hơn nhiều do có thể xảy ra phản ứng dây chuyền.
- Hàm delNode trả về giá trị 1, 0 khi hủy thành công hoặc không có X trong cây. Nếu sau khi hủy, chiều cao cây bị giảm, giá trị 2 sẽ được trả về:
- `int delNode(AVLTree &T, DataType X)`

Hủy một phần tử trên cây AVL

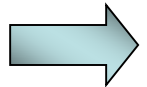
```
int delNode(AVLTree &T, DataType X)
{
    int res;
    if(T==NULL) return 0;
    if(T->key > X)
    {
        res = delNode (T->pLeft, X);
        if(res < 2) return res;
        switch(T->balFactor)
        {
            case LH: T->balFactor = EH; return 2;
            case EH: T->balFactor = RH; return 1;
            case RH: return balanceRight(T);
        }
    } // if(T->key > X)
    .....
}
```



delNode2

Hủy một phần tử trên cây AVL

```
int delNode(AVLTree &T, DataType X)
{
.....
    if(T->key < X)
    {
        res = delNode (T->pRight, X);
        if(res < 2)    return res;
        switch(T->balFactor)
        {
            case RH: T->balFactor = EH; return 2;
            case EH: T->balFactor = LH; return 1;
            case LH: return balanceLeft(T);
        }
    } // if(T->key < X)
.....
}
```



delNode3

Hủy một phần tử trên cây AVL

```
int delNode(AVLTree &T, DataType X)
{.....
    else //T->key == X
    { AVLNode* p = T;
        if(T->pLeft == NULL) { T = T->pRight; res = 2; }
        else if(T->pRight == NULL) { T = T->pLeft; res = 2; }
        else //T có đủ cả 2 con
        { res = searchStandFor(p,T->pRight);
            if(res < 2) return res;
            switch(T->balFactor)
            { case RH: T->balFactor = EH; return 2;
              case EH: T->balFactor = LH; return 1;
              case LH: return balanceLeft(T);
            }
        }
        delete p; return res;
    }
}
```

Tìm phần tử thể mạng searchStandFor

```
int searchStandFor(AVLTree &p, AVLTree &q)
```

```
//Tìm phần tử thể mạng
```

```
{  int   res;
    if(q->pLeft)
    {  res= searchStandFor(p, q->pLeft);
        if(res < 2)   return res;
        switch(q->balFactor)
        {  case LH: q->balFactor = EH; return 2;
            case EH: q->balFactor = RH; return 1;
            case RH: return balanceRight(T);
        }
    }  else
    {  p->key = q->key; p = q; q = q->pRight; return 2;
    }
}
```

Cây AVL

- Nhận xét:

- Thao tác thêm một nút có độ phức tạp $O(1)$.
- Thao tác hủy một nút có độ phức tạp $O(h)$.
- Với cây cân bằng trung bình 2 lần thêm vào cây thì cần một lần cân bằng lại; 5 lần hủy thì cần một lần cân bằng lại.

Cây AVL

- Nhận xét:

- Việc huỷ 1 nút có thể phải cân bằng dây chuyền các nút từ gốc cho đến phần tử bị huỷ trong khi thêm vào chỉ cần 1 lần cân bằng cục bộ.
- Độ dài đường tìm kiếm trung bình trong cây cân bằng gần bằng cây cân bằng hoàn toàn $\log_2 n$, nhưng việc cân bằng lại đơn giản hơn nhiều.
- Một cây cân bằng không bao giờ cao hơn 45% cây cân bằng hoàn toàn tương ứng dù số nút trên cây là bao nhiêu.